EECS708: Machine Learning

Density Estimation

Dr. Ioannis Patras
School of EECS

Slide thanks: Dr. Tim Hospedales

Course Context

- Supervised Learning
  - (Linear) regression
  - Logistic Regression (Classification)
  - Neural Networks
- Unsupervised
  - Clustering
  - Density Estimation
  - Dimensionality reduction (partial)
- Advanced topics
  - Deep Learning, Convolutional Neural Networks
  - Ensemble Learning

Overview

- Overview: What is Density estimation about?
- Gaussian Density Estimation
  - ML fitting of a Gaussian
  - Overfitting / singular covariances
- Gaussian Mixture models
  - ML fitting with Expectation Maximization
  - Choosing the number of Gaussians
- Some applications
  - Anomaly detection
  - Bayesian classification

Probability & Density Estimation

Three common probability distributions

- Binary variables: Bernoulli
  - x is 1,0 (Heads or Tails).
  - u (probability of Heads) from 0 to 1.

$$p(x;u) = u^x (1-u)^{(1-x)}$$

- Categorical variables: Multinomial
  - x is 1-of-K encoding.
  - ui (probability of outcome i) from 0 to 1. ui's sum over I equals to 1.

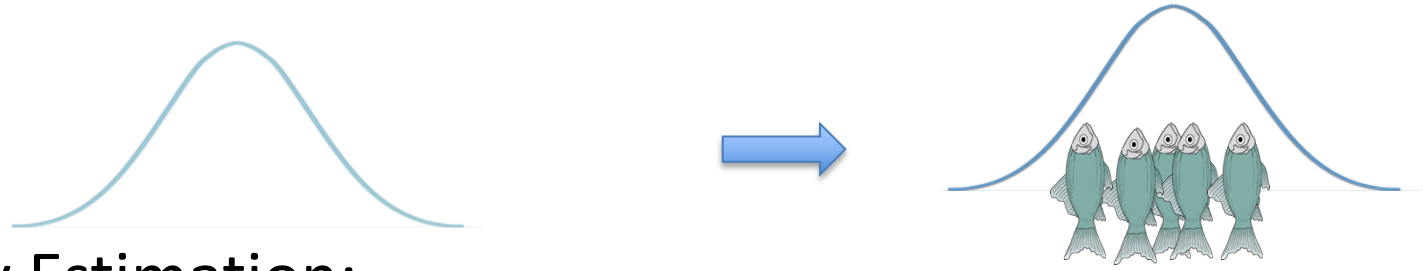
$$p(\mathbf{x};\mathbf{u}) = \prod u_i^{x_i}$$

- Continuous variables: Gaussian
  - x is real vector. u is a real vector. S is a matrix.

$$p(\mathbf{x};\mathbf{u},S) = \frac{1}{Z}\exp\left(-\frac{1}{2}(\mathbf{x}-\mathbf{u})^T S^{-1}(\mathbf{x}-\mathbf{u})\right)$$

Probability & Density Estimation

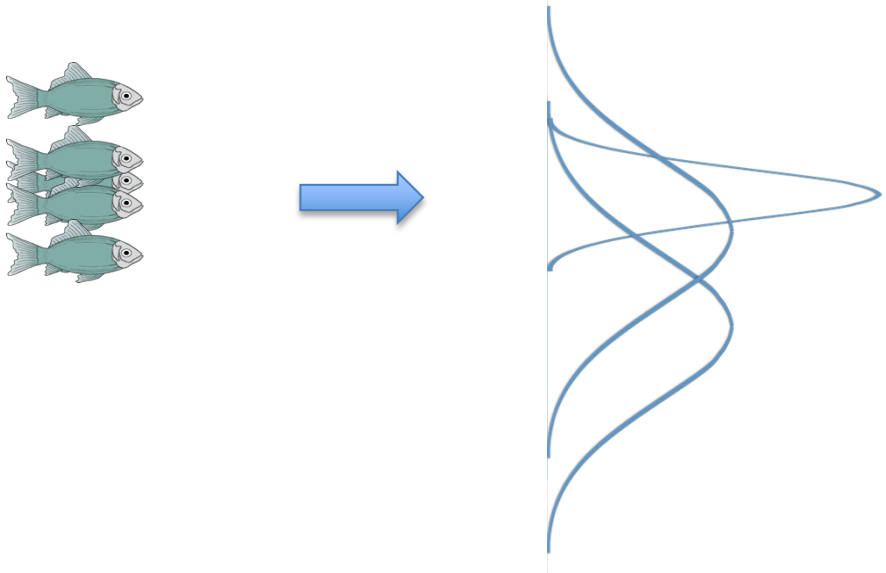
- Generative Perspective:
  - Distributions tell us what data to expect according to specified parameters
  - E.g., Biased coin (u=3/4). → Expect H,H,H,T
  - E.g., Mean & var of fish length → Expect



- Density Estimation:
  - Ask what probability distribution was responsible for specified data?

Probability & Density Estimation

- Density Estimation: Ask what probability distribution was responsible for specified data?
  - H,H,H,T,T → Is u=100/100, 75/100, 25/100, etc more likely?



- There are simple exact solutions for the best estimates of binary, categorical, and Gaussian distributions given data

Probability & Density Estimation

- Density Estimation: Ask what probability distribution was responsible for specified data?
  - There are simple exact solutions for the best estimates of binary, categorical, and Gaussian distributions given data

$$p(x;u) = u^{x}(1-u)^{(1-x)} \quad \Rightarrow \quad u = \frac{1}{N}\sum_{i} x_i$$

$$p(\mathbf{x};\mathbf{u}) = \prod u_k^{\,x_k} \quad \Rightarrow \quad u_k = \frac{\sum_i x_{ik}}{\sum_i \sum_k x_{ik}} = \frac{N_k}{N}$$

$$p(\mathbf{x};\mathbf{u},S) = \frac{1}{Z}\exp\left(-\frac{1}{2}(\mathbf{x}-\mathbf{u})^T S^{-1}(\mathbf{x}-\mathbf{u})\right) \quad \Rightarrow \quad \mathbf{u} = \frac{1}{N}\sum_i \mathbf{x}_i \quad S = \frac{1}{N}\sum_i (\mathbf{x}-\mathbf{u})(\mathbf{x}-\mathbf{u})^T$$

Summary

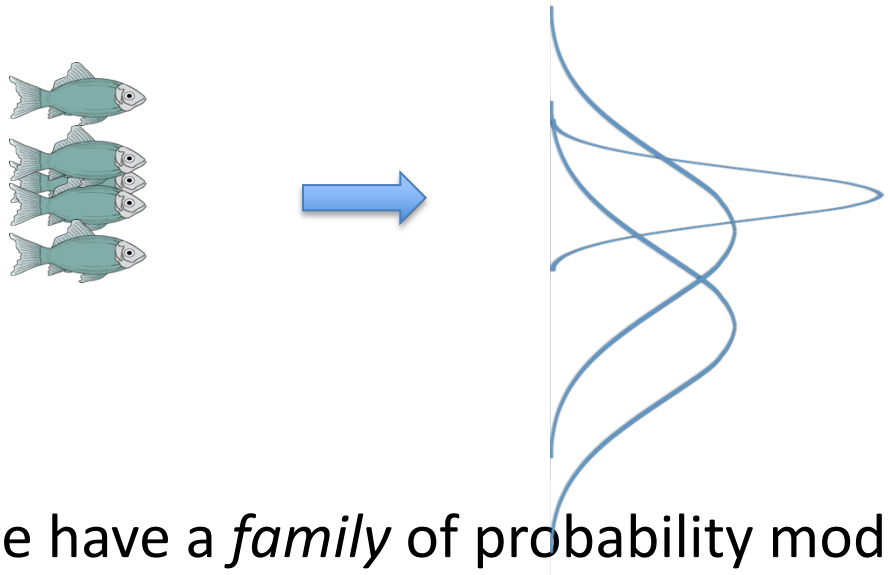
- Density estimation: which probability distribution generated the observed data?
- Closed form formulas for common distributions – choose the right one!

Overview

- Overview: What is Density estimation about?
- Gaussian Density Estimation
  - ML fitting of a Gaussian
  - Overfitting / singular covariances
- Gaussian Mixture models
  - ML fitting with Expectation Maximization
  - Choosing the number of Gaussians
- Some applications
  - Anomaly detection
  - Bayesian classification

Fitting a single Gaussian

- Density Estimation: What probability distribution was responsible for specified data?



- Formally, we have a *family* of probability models for $x \sim p(x;\theta)$, where θ denotes an unknown parameter vector.

Let $\mathrm{X} = \{x_1, x_2, \ldots, x_N\}$ we wish to fit a density

$$p(x;\theta) = N(x;\mu,\Sigma), \qquad i.e., \theta = \{\mu, \Sigma\}$$

$$x_n \in \mathbb{R}^d, \mu \in \mathbb{R}^d, \Sigma \in \mathbb{R}^{d\times d}$$

Maximum Likelihood estimation

Assuming samples are independent, the data likelihood is

$$\mathrm{p}(x_1, x_2, \dots, x_N; \theta) = \prod_{\mathrm{n}=1}^{\mathrm{N}} p(x_n; \theta) = \mathrm{L}(\theta)$$

We minimise the negative loglikelihoood:

$$E = -\ln \sum_{n} L(\theta) = -\sum_{\mathrm{n}} \ln p\,(x_n|\theta)$$

Case study: ML fitting of a single Gaussians

In one dimension:
Let $X = \{x_1, x_2, \dots, x_N\}$ we wish to fit a density $p(x; \mu, \sigma)$
$= N_x(\mu, \sigma)$

$$E = -\ln L(\theta) = \sum_{\mathrm{n}} \left(\frac{(x_n - \mu)^2}{2\sigma^2} + \ln \sigma + \frac{1}{2} \ln 2\pi \right).$$

Estimate $\boldsymbol{\mu}$: Set derivative to zero

$$\left. \frac{\partial E}{\partial \mu} \right|_{\hat{\boldsymbol{\mu}}} = -\sum_{\mathrm{n}} \frac{(x_n - \hat{\mu})}{\sigma^2} = 0$$

$$\rightarrow \qquad \hat{\mu} = \frac{1}{N} \sum_{n} x_n \qquad \text{(sample mean!)}$$

Case study: ML fitting a single Gaussian

To solve for ML estimate of σ we use the fact that we already know $\hat{\mu}$:

$$\frac{\partial\, E}{\partial \sigma} = \sum_n \left(\frac{1}{\sigma} - \frac{(\mathbf{x}_n - \hat{\boldsymbol{\mu}})^2}{\sigma^3} \right)$$

Equating to zero gives:

$$\sigma^2 = \frac{1}{N} \sum_n (\mathbf{x}_n - \hat{\mu})^2 \qquad \text{(sample variance)}$$

Recall that the sample variance is a biased estimate for variance.

However such ML estimates are consistent and asymptotically optimal.

ML fitting a single Gaussians

In higher dimensions $p(x|\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)}$

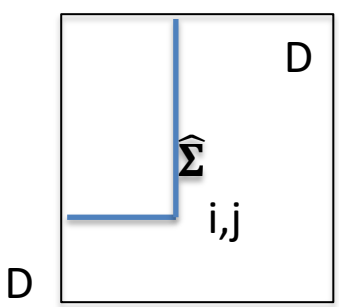
we get the similar results of:

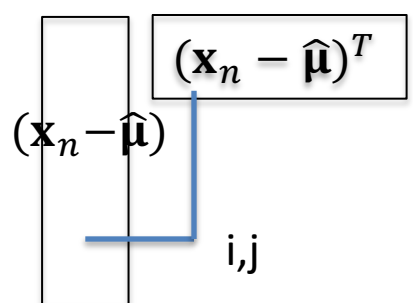
ML estimate for mean $\hat{\boldsymbol{\mu}}$:

$$\hat{\boldsymbol{\mu}} = \frac{1}{N}\sum_n \mathbf{x_n} \qquad \text{(sample mean)}$$

and ML estimate for covariance matrix:

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_n (\mathbf{x}_n - \hat{\boldsymbol{\mu}})(\mathbf{x}_n - \hat{\boldsymbol{\mu}})^T \qquad \text{(sample covariance)}$$

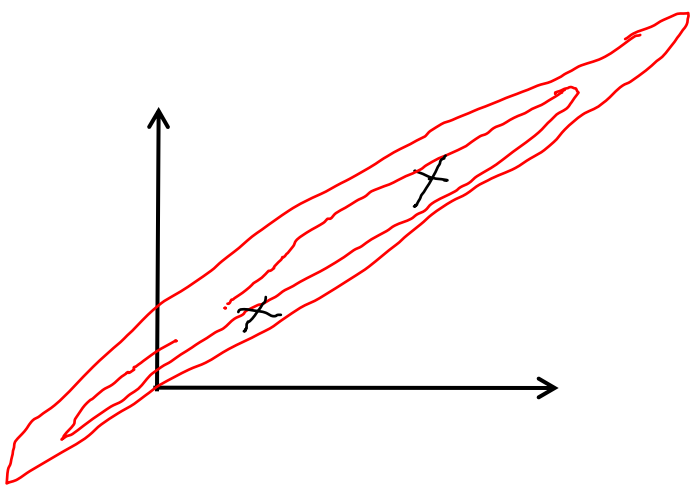




Overfitting in ML density estimation

Problem: In the case that the data dimensionality is higher than the data samples the covariance matrix is singular.

Non invertible, zero determinant.



$$p(x|\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)}$$

$$\widehat{\mathbf{\Sigma}} = \frac{1}{N}\sum_n (\mathbf{x}_n - \widehat{\boldsymbol{\mu}})(\mathbf{x}_n - \widehat{\boldsymbol{\mu}})^T$$

Overfitting in ML density estimation

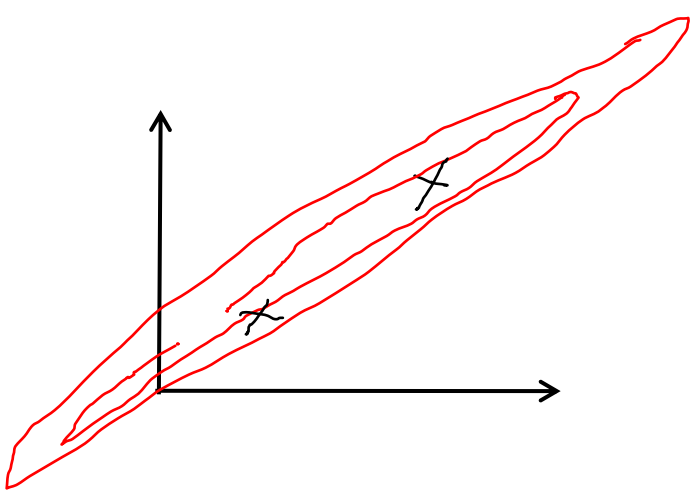
$$p(x|\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)}$$

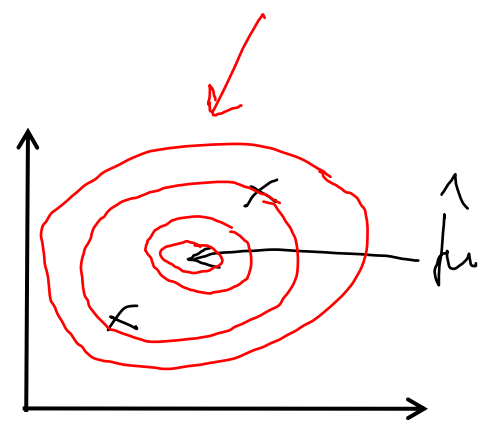
1. Put constraints in the form of the covariance matrix. E.g.

$$\Sigma = \sigma I = \begin{bmatrix} \sigma & & 0 \\ & \sigma & \\ 0 & & \sigma \end{bmatrix}$$

$$\sigma^2 = \frac{1}{N}\sum_n (x_n - \hat{\mu})^T(x_n - \hat{\mu})$$

Concentric circles





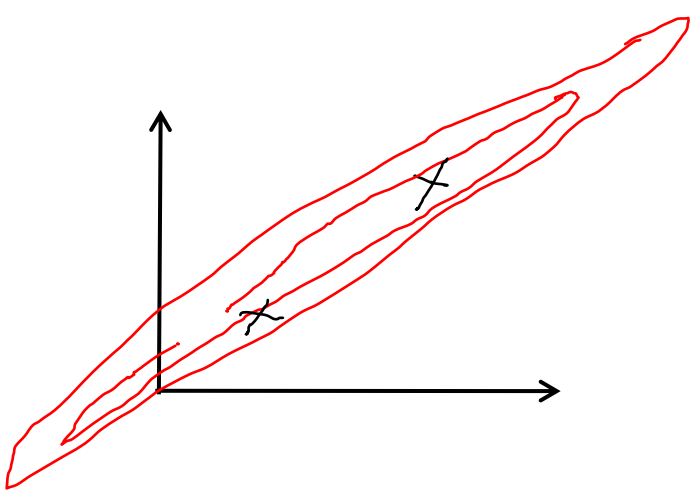
Overfitting in ML density estimation

$$p(x|\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)}$$

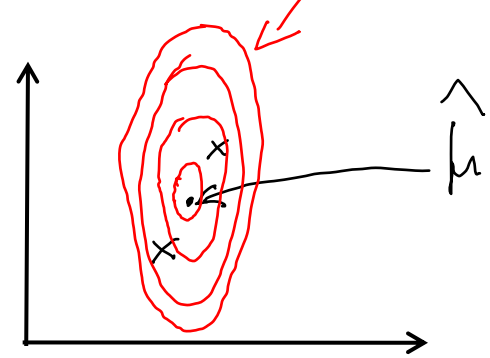
2. Put constraints in the form of the covariance matrix. E.g.

$$\Sigma = \begin{bmatrix} \sigma_1 & \dots & \sigma_d \end{bmatrix} I = \begin{bmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_j & \\ & & & \ddots \\ & & & & \sigma_d \end{bmatrix}$$

$$\sigma_j{}^2 = \frac{1}{N}\sum_n \left(x_n(j) - \hat{\mu}(j)\right)^2$$



Axes aligned ellipse



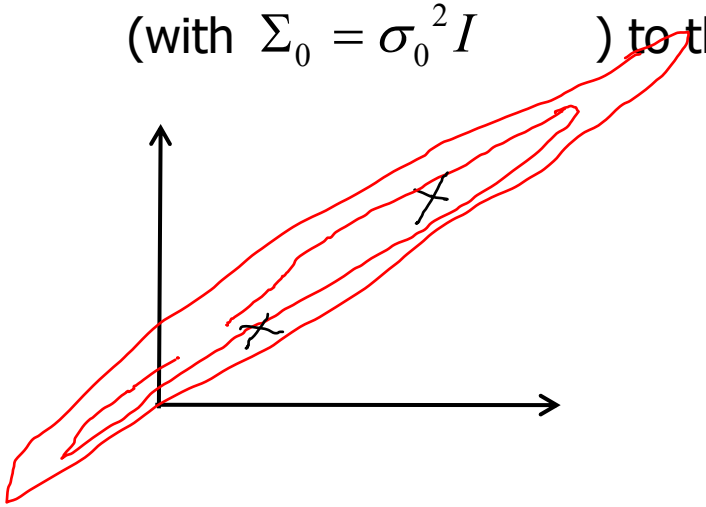
Overfitting in ML density estimation

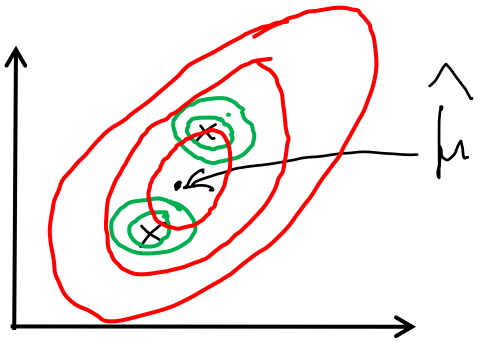
$$p(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \mid \Sigma \mid^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

2. Regularise the covariance matrix by adding a diagonal matrix $\Sigma_0 = \sigma_0{}^2 I$

$$\hat{\Sigma} = \frac{1}{N}\sum_n (x_n - \hat{\mu})(x_n - \hat{\mu})^T + \sigma_0{}^2 I = \begin{bmatrix} \sum_n (x_n(1) - \hat{\mu}(1))^2 + \sigma_0 & & \\ & \ddots & \\ & & \sum_n (x_n(d) - \mu_n(d))^2 + \sigma_0 \end{bmatrix}$$

Leads to "thicker" Gaussian. Equivalent to adding zero mean Gaussian noise (with $\Sigma_0 = \sigma_0{}^2 I$) to the original dataset.





Summary

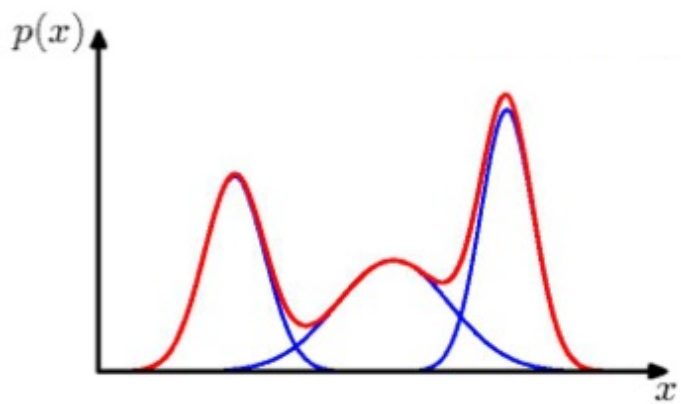
- Fitting a Gaussian with a ML estimation
  - Define the objective (negative loglikelihood)
  - Set derivatives to zero
- Danger of overfitting
  - Constraints on the covariance matrix

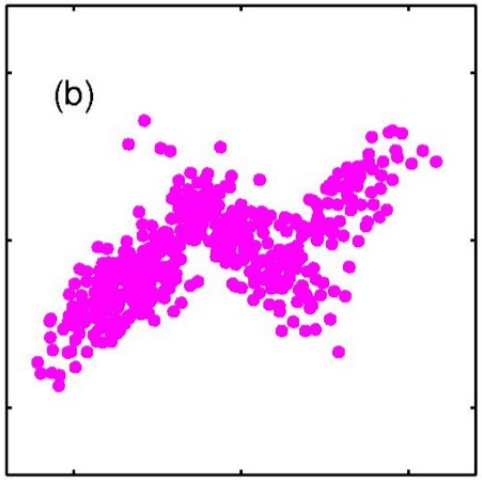
Overview

- Overview: What is Density estimation about?
- Gaussian Density Estimation
  - ML fitting of a Gaussian
  - Overfitting / singular covariances
- Gaussian Mixture models
  - ML fitting with Expectation Maximization
  - Choosing the number of Gaussians
- Some applications
  - Anomaly detection

Gaussian Mixture Models

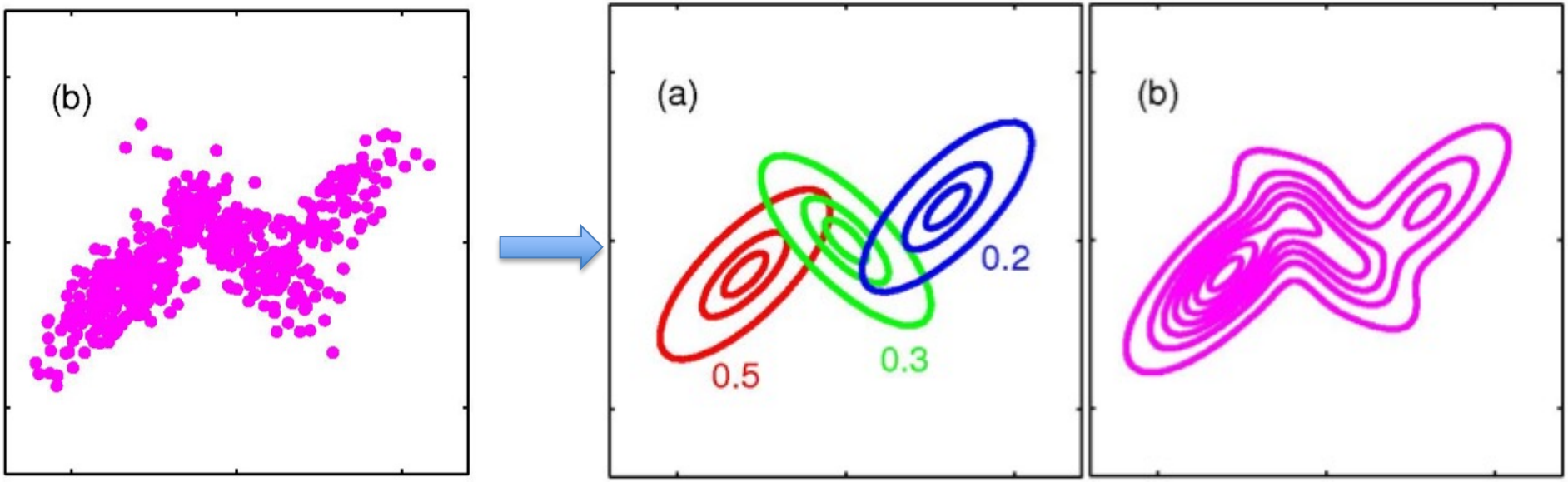
- Tough data $\mathrm{p}(x; \theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1\ldots K}) = \sum_{k=1}^{K} \pi_k N(x; \mu_k, \Sigma_k)$
  - K-means has problems
  - Single Gaussian doesn't fit it well
- GMM solution:
  - Explain as: Weighted sum of K Gaussian densities





Gaussian Mixture Models

- GMM density estimation: $\mathrm{p}(x;\theta\ ) = \sum_{k=1}^{K} \pi_k N(x;\mu_k,\Sigma_k)$
  - Explain as: Weighted sum of K Gaussian densities
- Example problem and solution



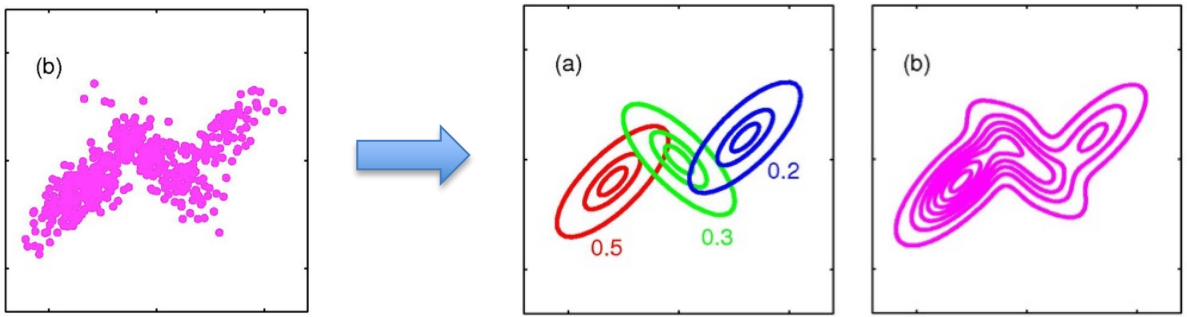
- … But what algorithm can obtain these solutions?

Gaussian Mixture Models: Solution

- Optimization Criterion: Maximum Likelihood

$$L(X;\theta) = \sum_{n=1}^{N}\sum_{k=1}^{K} \pi_k N(x_n;\mu_k,\Sigma_k)$$

- Solution?
  - If we knew which points belong to which clusters, we know how to fit Gaussians (Density Estimation: Gaussian)
  - If we knew which points belong to which cluster, we know the relative size of each (Density Estimation: Multinomial)
  - If we knew the Gaussians, we could find out which points belonged to which clusters (Bayes Theorem)

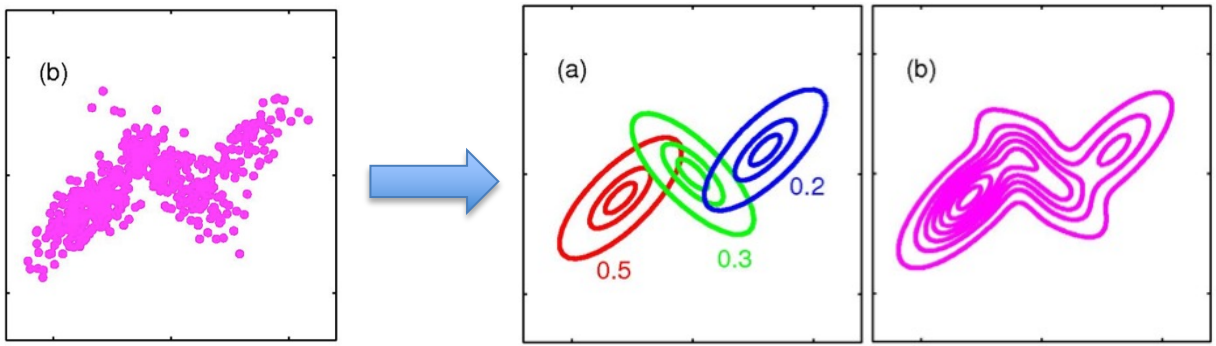


Gaussian Mixture Models: Solution

- Optimization Criterion: Maximum Likelihood

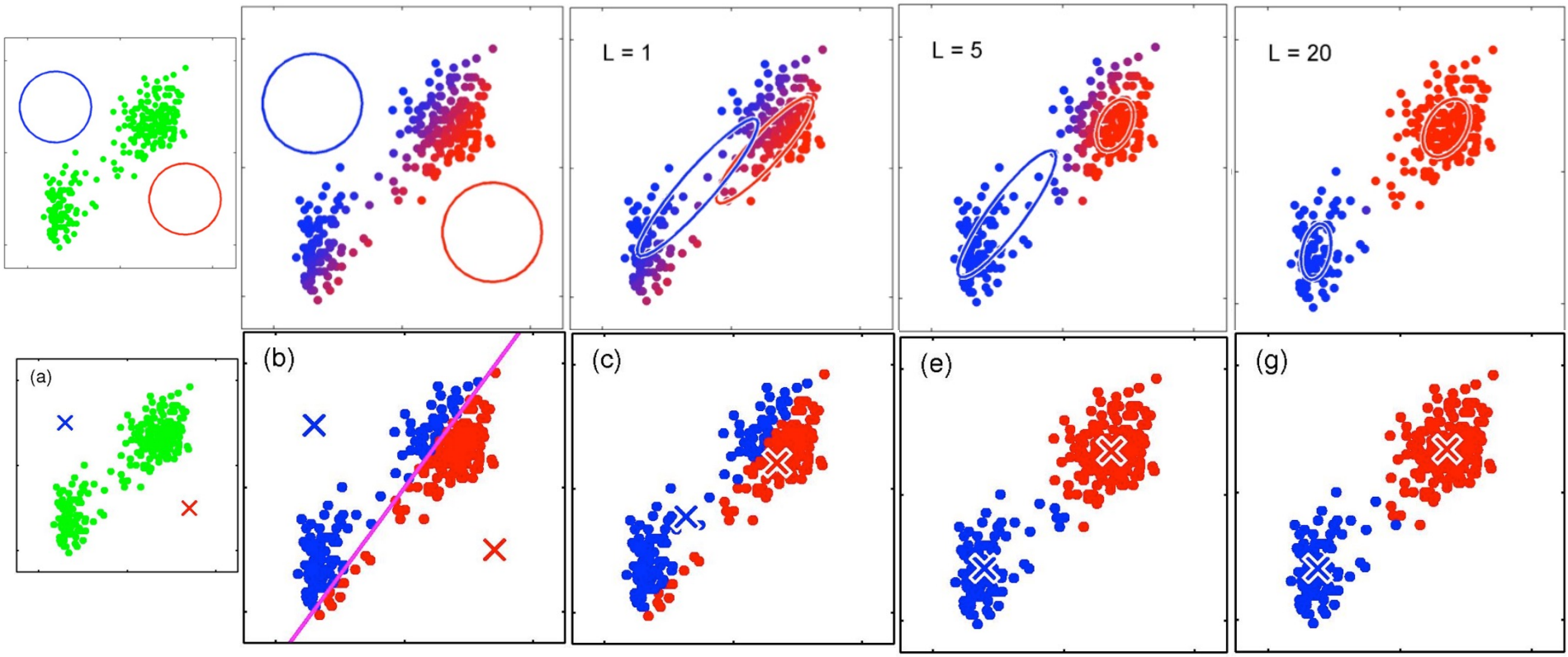
$$L(X;\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1\ldots K}) = \sum_{n=1}^{N}\sum_{k=1}^{K} \pi_k N(x_n; \mu_k, \Sigma_k)$$

- EM iterative Algorithm Solution (at iteration i)
  - E: Given parameters $\theta^{(i-1)}$, infer how likely each point to each cluster. (calculate auxiliary variables $q_{n,k}$)
  - M: Given soft assignments, update Gaussians and cluster prior. (calculate $\theta^{(i)}$)



GMM: EM Solution Example

What contrasts do you see?



EM for Mixture of Gaussians (MoG)

$$\mathrm{p}(x; \theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1\ldots K}) = \sum_{k=1}^{K} \pi_k N(x; \mu_k, \Sigma_k)$$

Find parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1\ldots K}$

Data structures:

N is #dinemsions, D is #samples, K is #Gaussians

X (data matrix)	NxD	reals
u = $[\mu_k]$, (Gaussian centres)	KxD	reals
SS = $[\Sigma_k]$, (Gaussian covariances	KxDxD	reals
P = $[\pi_k]$, (mixture coefficients)	K	reals
q (Soft assignments matrix)	NxK	reals

Algorithm:

Initialize θ

Iterate

- "E-step": Softly assign points to Gaussians (fix u, optimize for q)
- "M-step": Find new means/covariances and mixture weights of each Gaussian (fix q, optimize for P,u,SS)

EM iterative update

Expectation − Maximization (beginning with initial values $\hat{\theta}^{(0)}$)

E − step

calculate: $q_{n,k} = p(c_k | x_n, \hat{\theta}^{(i)})$ for all k and n

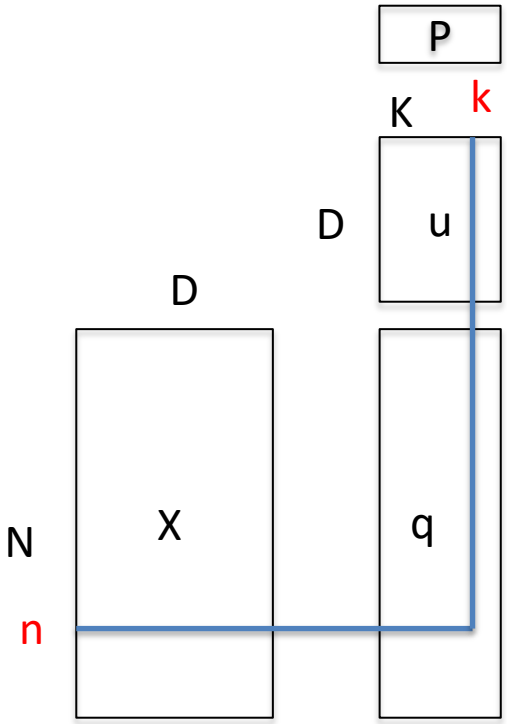
M−step

maximise the weighted log likelihood:

$$\hat{\theta}_k^{(i+1)} = \underset{\theta}{\operatorname{argmax}} \sum_n q_{n,k} \ln p\,(x_n | c_k, \theta_k)$$

for all k

A converged solution is a stationary point (e.g. local maximum) of the likelihood



Algorithm:

– "E-step": Softly assign points to Gaussians (fix u, optimize for q)

```
For 1< i < N
        for 1<k<K
                q[n,k] = gaussian(X[n,:], u[k,:], SS[k,:,:])P[k];
        end
         q[n,:] = q[n,:] / sum(q[n,:]);   % Normalise q along rows
end
```

EM algorithm for MoGs I

EM algorithm to estimate MoG parameters.

Let the ith estimate for $\pi_k^{(i)} = p(c_k)$:

- E−step : Here we calculate $q_{n,k} = p(c_k|x_n, \theta^{(i)})$ by using

$$p(c_k|x_n, \theta^{(i)}) = p(x_n|c_k, \theta^{(i)})p(c_k)/p(x_n|\theta^{(i)})$$

$$= p(x_n|c_k, \theta^{(i)})p(c_k)/\sum_j p(x_n|c_j, \theta^{(i)})p(c_j)$$

$$= p\left(x_n\middle|c_k, \theta^{(i)}\right)\pi_k^i / \sum_j p(x_n|c_j, \theta^{(i)})\pi_k^i$$

Where $p(x_n|c_k, \theta^{(i)}) = \frac{1}{{\Sigma_k}^{d/2}} \exp(-\frac{1}{2}(x_n - \mu_k)^T \Sigma_k^{-1}(x_n - \mu_k))$

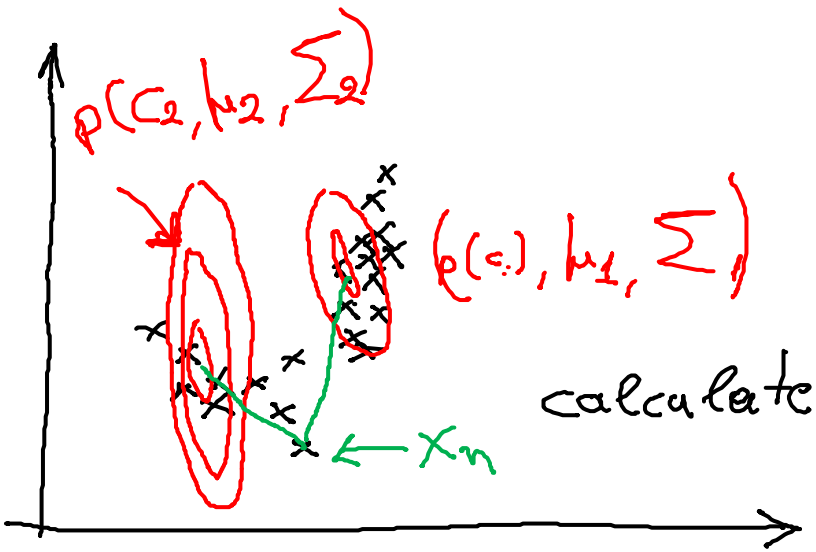
EM iterative update (E-step)

Expectation - Maximization (beginning with initial values $\hat{\theta}^{(0)}$)

E – step calculate: $q_{n,k} = p(c_k \mid x_n, \hat{\theta}^{(i)})$ for all k and n

M-step maximise the weighted log likelihood:

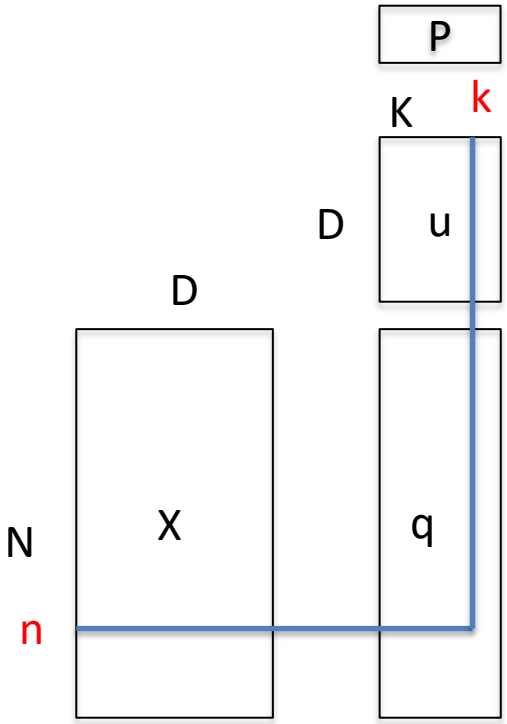
$$\hat{\theta}_k^{(i+1)} = \underset{\theta}{\operatorname{argmax}} \sum_n q_{n,k} \ln p(x_n \mid c_k, \theta_k) \qquad \text{for all } k$$



calculate $q_{n1} = p(c_1 \mid x_n, \theta)$

$$= \frac{p(x_n \mid c_1, \theta)\, p(c_1)}{p(x_n \mid \theta)}$$

$$= \frac{p(x_n \mid c_1, \theta) \cdot p(c_1)}{\sum_k p(x_n \mid c_k, \theta) \cdot p(c_k)}$$



Algorithm:

- M-step": Find new means/covariances and mixture weights of each Gaussian (fix q, optimize for P,u,SS)

```
for 1<k<K
        u[k,:]  = sum(q[:,K]*X) / sum(q[:,K]);
        SS[k,:,:] = sum(q[:,K]*X*X^T) / sum(q[:,K]);
        P[k]= sum(q[:,k]) / N
 end
```

EM algorithm for MoGs

- M−step :

1. $\mu_{\mathrm{k}}^{(i+1)} = \dfrac{\sum_n q_{n,k} x_n}{\sum_n q_{n,k}}$ (weighted mean)

2. $\Sigma_{\mathrm{k}}^{(i+1)}$

$= \dfrac{\sum_n q_{n,k}(x_n - \mu_{\mathrm{k}}^{(i+1)})(x_n - \mu_{\mathrm{k}}^{(i+1)})^T)}{\sum_n q_{n,k}}$ (weighted covariance)

And we can estimate the priors $p(c_k)$ for each class c_k using the following update:

3. $\pi_k^{(i)} = \dfrac{1}{N} \sum_n q_{n,k}$

All the variables can be updated as weighted averages!

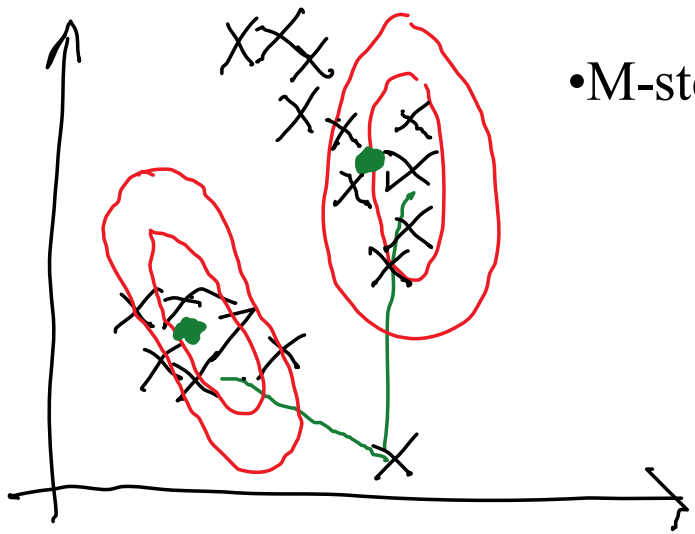
EM iterative update (M-step)

Expectation - Maximization (beginning with initial values $\hat{\theta}^{(0)}$)

E – step calculate: $q_{n,k} = p(c_k \mid x_n, \hat{\theta}^{(i)})$ for all k and n

M-step maximise the weighted log likelihood:

$$\hat{\theta}_k^{(i+1)} = \underset{\theta}{\operatorname{argmax}} \sum_n q_{n,k} \ln p(x_n \mid c_k, \theta_k) \qquad \text{for all } k$$



•M-step : $\mu_k^{(i+1)} = \sum_n q_{n,k} x_n \Big/ \sum_n q_{n,k}$ (weighted mean)

GMM: Limitations

- Converges to local optima only
  - Can also do multiple restarts and pick the best
- Picking K is still an issue
- Cost $O(NKD^2)$
  - Data requirements >> O(D) due to covariance matrix
    - (Estimating the shape information of each)

GMM versus K-Means

K-means

Algorithm:

- For i=1:N
  - c_i=Index i of the cluster centroid closest to x_i
- For k=1:K
  - u_k = average of points assigned to k

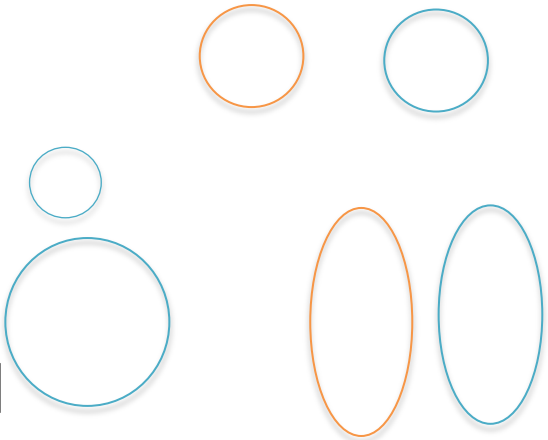
GMM

Algorithm:

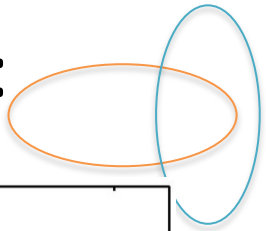
- For i = 1 : N
  - Probability $p(k|x_i)$ of belonging to each cluster k
- For k=1:K
  - Fit the Gaussian $N(u_k,S_k)$ given probabilities $p(k|x_i)$
  - Fit the cluster prior $p(k)$ given probabilities $p(k|x)$.

GMM: A clustering perspective

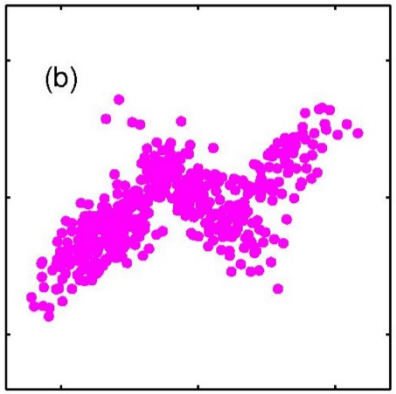
- K-means works if:
  - Clusters are spherical
  - Clusters are well separated
  - Clusters are of similar spread
  - Clusters have same number of points



- K-means won't work for data like this:



- Motivate:
  - Mixture of Gaussians algorithm



GMM versus K-Means

K-means

Assumptions

- All clusters same size
- All clusters spherical
- Clusters are sharply peaked
- Hard assign points to clusters
- Optimize:
  - Sum Squared Dist of points to clusters

$$E_{KM}(D, c_{1:N}, \mu_{1:K}) = \sum_{i=1}^{N} \left(x_i - \mu_{c_i}\right)^2$$

GMM

Assumptions

- Cluster k of size π_k
- Clusters of shape S
- Clusters have spread S
- Soft assign points to clusters
- Optimize:
  - Log-likelihood of data

$$L_{GMM}(D; \boldsymbol{\pi}, \mathbf{u}, S) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k N(\mathbf{x}_i; \mathbf{u}_k, S_k)$$

GMM versus K-Means

K-means

- Pick K is non-trivial
- Only local optimization
- Cost:
  - CPU: O(NKD)

GMM

- Pick K is non-trivial
- Only local optimization
- Cost:
  - CPU: O(NKD^2):
  - Data: >> O(D)

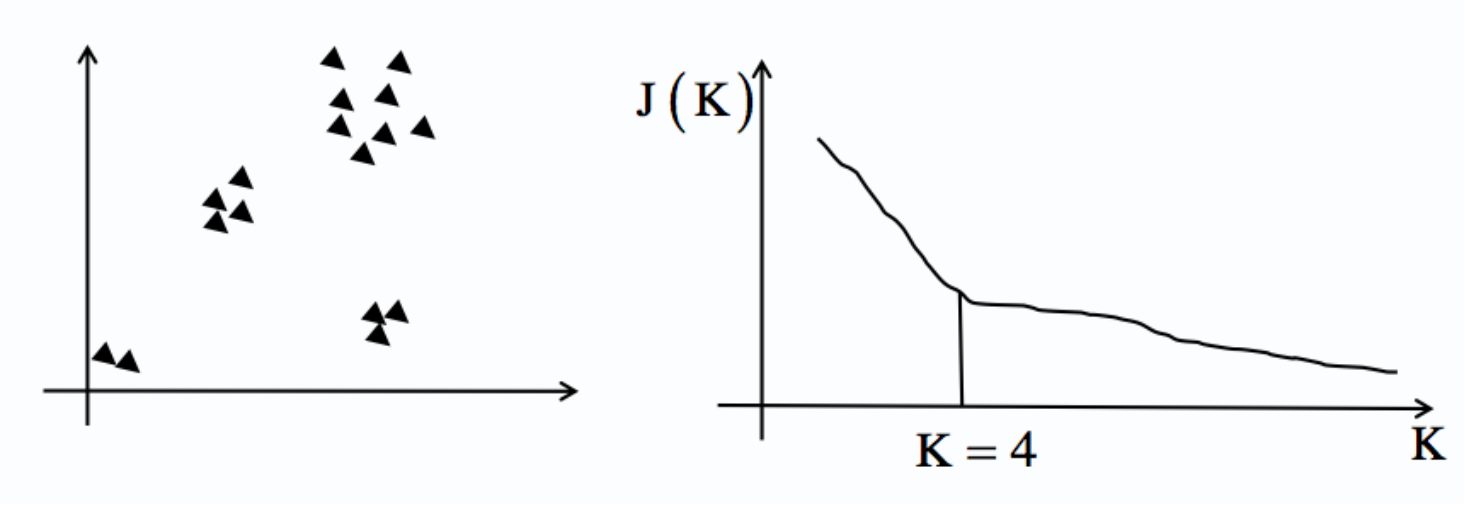
$$E_{KM}(D, c_{1:N}, \mu_{1:K}) = \sum_{i=1}^{N} \left(x_i - \mu_{c_i}\right)^2$$

$$L_{GMM}(D; \boldsymbol{\pi}, \mathbf{u}, S) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k N(\mathbf{x}_i; \mathbf{u}_k, S_k)$$

Choosing Number of Clusters

For K-means and GMM

- 1. Elbow Method
  - Plot $E_{KM/GMM}$ as a function of k, and choose the elbow point.



- 2. Present results to end users see what they prefer
  - Broader or more specialized groups

Choosing Number of Clusters

For K-means and GMM

- 3. Cross-validation
  - For K = 1…Large
    - Learn GMM on a train set.
    - Evaluate Quality(K) = quality on validation set.
  - Pick K with the highest validation set quality.

Choosing Number of Clusters

For K-means and GMM

- 4. BIC/AIC Criterion
  - (ML people: An approximation to the integration required in the Bayesian model selection)
  - Adds a penalty to the cost that penalises more complex models.
    - P: Number of parameters in model. N: Number of data points.
  - Evaluate modified cost E^K_{BIC} for many values of K.
  - Pick the K with best cost

$$E^K_{BIC} = E^K + \frac{p}{2} logN$$

$$E_{KM}(D, c_{1:N}, \mu_{1:K}) = \sum_{i=1}^{N} \left(x_i - \mu_{c_i}\right)^2$$

$$E_{GMM}(D; \boldsymbol{\pi}, \mathbf{u}, S) = -\log \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k N(\mathbf{x}_i; \mathbf{u}_k, S_k)$$

Summary

- Fitting a mixture of Gaussian with ML estimation
  - Expectation Maximization
  - Links/comparison to K-means
- How to choose the number of Gaussians.

Overview

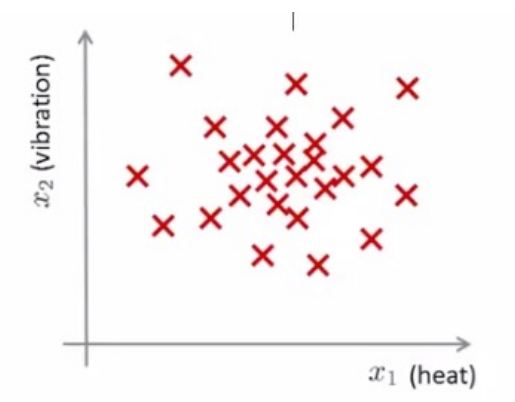
- Overview: What is Density estimation about?
- Gaussian Density Estimation
  - ML fitting of a Gaussian
  - Overfitting / singular covariances
- Gaussian Mixture models
  - ML fitting with Expectation Maximization
  - Choosing the number of Gaussians
- Some applications
  - Anomaly detection

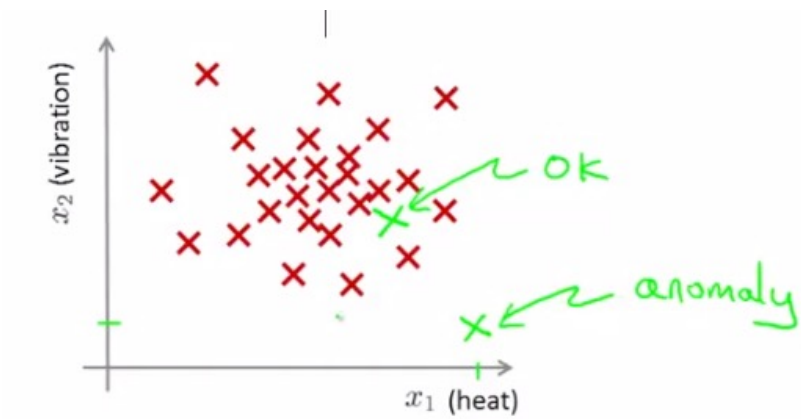
Anomaly Detection: Motivation

- Sometimes you are interested in finding unusual items
  - “Anomalies”, “Outliers”
- …As a pre-processing step
  - E.g., algorithms that use Sum-Squared objectives are not robust to outliers. Use outlier detection first to find and discard such rows
- …As an end goal.

Anomaly Detection: Example

- Example: Manufacturing Quality Control: Aircraft Engines
  - x_1 = heat, x_2 = vibration.





Anomaly Detection Algorithm

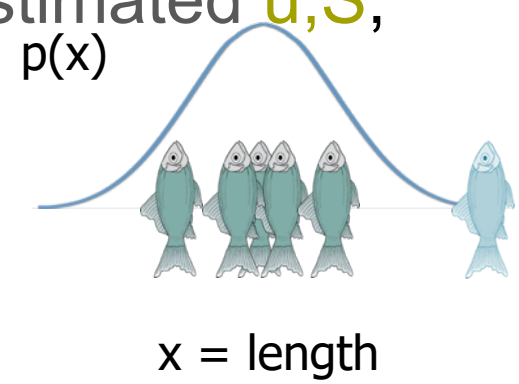
- Algorithm:
  - Read in normal training data, {x}
  - Compute the Gaussian (u,S) that best explains the data {x}

$$u = \frac{1}{N}\sum_n x_n \qquad S = \frac{1}{N}\sum_n (x_n - u)(x_n - u)^T$$

$$P(x; u, S) = \frac{1}{Z}\exp(-\frac{1}{2}(x-u)^T S^{-1}(x-u)$$

  - Given a new example x and estimated u,S, compute p(x)
  - If p(x)<T, then Anomaly

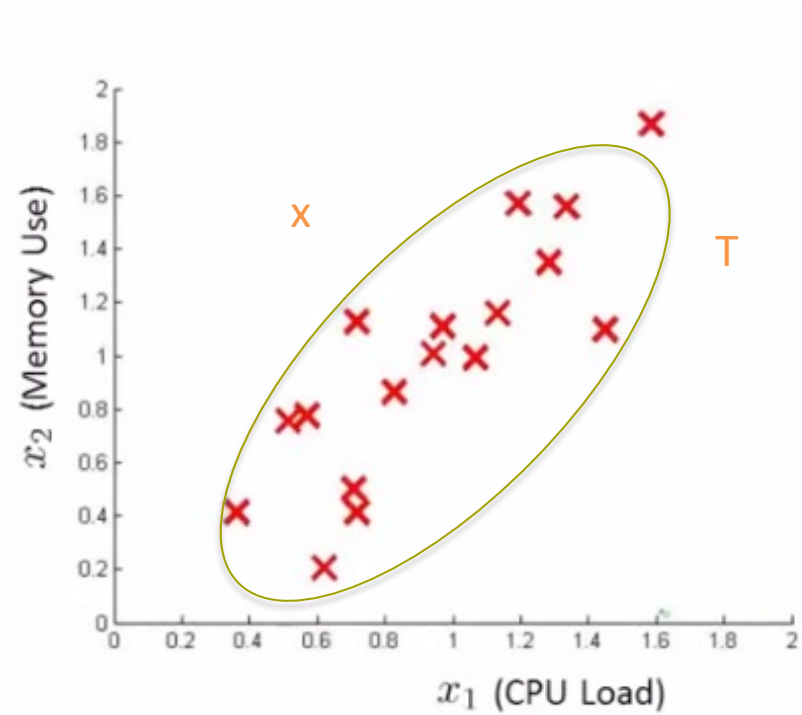
    else Ok



Anomaly Detection Example: Data Center: Multivariate

Algorithm:

Anomaly if p(x)<T



Gaussian Classifiers

- Assume some training data $X = \{(x_i, y_i\}$.
  - Produce a model y=f(x*) that can classify new data

$$P(y = j|x) = \frac{p(x|y = j)p(j)}{p(x)}$$

- Assign the new data point to class j iff

$$P(y = j|x) \geq P(y = i|x) \forall i \neq j \quad \Rightarrow$$

$$p(x|y = j)p(j) \geq p(x|y = i)p(i) \forall i \neq j$$

Gaussian Classifiers

- Assign the new data point to class j iff

$$p(x|y=j)p(j) \geq p(x|y=i)p(i) \forall i \neq j$$

- How to get *p(x|y=i)*? Maximum Likelihood Estimation:
  - Fit a multivariate normal to $\{x_i\}_i$ where $y_i=1$
  - Fit a multivariate normal to $\{x_i\}_i$ where $y_i=2$

Other Classifiers…

- Change the likelihood according to the data type:
- For continuous data: *p(x|w)* is commonly Gaussian / Mixture of Gaussians. x is a d dimensionsal vector

$$u = \frac{1}{N}\sum_n x_n \qquad S = \frac{1}{N}\sum_n (x_n - u)(x_n - u)^T$$

- For discrete data:
  - *Binary case: p(x|w)* is Bernoulli, $x_n \in \{0,1\}$ $\qquad \theta = \frac{1}{N}\sum_n x_n$
  - K-class case: *p(x|w)* is multinomial $x_n \in \{0,1\}^K$

$$w_{jk} = N_{jk}\sum_j x_{jk} \qquad N_{jk} = \sum_i I(x_{ik} = j)$$

  - Common for: spam classifiers, sentiment recognition, etc.

$$N_{jk} = \sum_i I(x_{ik} = j)$$

Summary

- Anomaly / outlier detection: Fit a probability density and detect outliers where probability is below a threshold.
- Bayesian classifiers: Fit a probability density to each class and classify according to the posterior of the class.
- Choose the distribution based on the data!